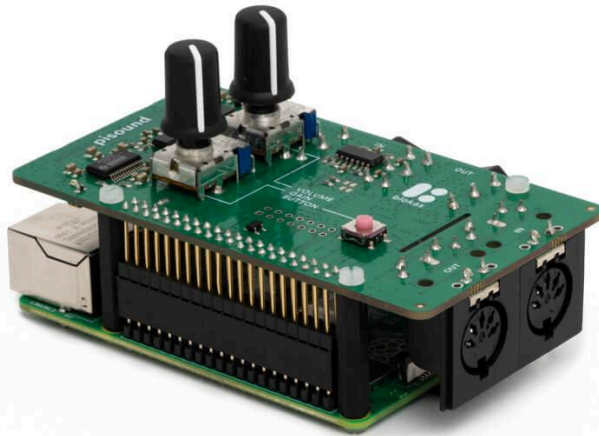


LivePi VideoGlitcher HLD

We want to build a visuals generator that runs on a Raspberry Pi 4 or 5 using the [Pisound](#) hardware interface. It needs to be compact, professional feeling, easy to setup, and simple to deploy in a live performance setting. It must react to connected audio and MIDI devices for tempo and other cues.



Idea Dump

- Load video clips onto the SD card
- MIDI input provides tempo clock
- Audio Input is used for generative video effects
- Button changes scenes
- Left Knob has middle detent
 - Could be used for positive/negative value
- Right Knob is a standard pot
 - Could control intensity

Discussion with ai about what I'm looking for using Ron Sheridan as an example.

Rob Sheridan's style relies on image destruction, not image generation. It thrives on analog signal degradation, broken horizontal sync, VHS tracking tears, RGB color splitting (chromatic aberration), and harsh blocky digital compression artifacts.

Since you are already filming gorgeous, authentic analog textures from your CRT TV setup as your base video clips, the Raspberry Pi's job shouldn't be to draw shapes—it should be to vandalize your video frames in tight synchronization with your hardware MIDI clock.

1. The Strategy: GLSL Shaders over openFrameworks or Python

To get real-time 60 FPS glitching on a Raspberry Pi 4 or 5, you want to process the videos using the Pi's GPU. The best tool for this is GLSL (OpenGL Shading Language) fragment shaders.

Instead of writing complex code architectures, a shader takes your video frames and recalculates the pixels instantly.

For Remote Collaboration: This is a dream. Shaders are just single text files (e.g., glitch.frag). Your friend can write and test a heavy VHS-tear shader on their laptop using a free app like Koreographer or an online GLSL sandbox, text the code to you, and it will execute identically on your Raspberry Pi.

You can load these shaders and your video files into a lightweight creative coding environment on the Pi like openFrameworks (C++) or Python (using PyOpenGL / Pi3D). openFrameworks has a legendary community-made add-on called ofxGlitch which specifically emulates broken buffers, bit-twiddling, and texture freezing.

2. Coding for the MIDI Clock Pulse

MIDI Clock sends 24 pulses per quarter note (PPQN). It doesn't tell the Pi the tempo directly; it just sends a steady stream of rhythmic triggers.

You or your friend will write a simple script (Python with python-rtmidi or openFrameworks) to listen to that port. You can use a basic pulse counter to map visual chaos directly to the beats of your dawless rig:

The 24-Pulse Counter:

Every time the Pi counts 24 pulses, a quarter-note beat has happened. On beat 1, you can randomly slice the video timeline. On beat 2, you can trigger a massive horizontal sync tear. On beat 4, you can flash a heavy color-inversion block.

MIDI CC Modulations:

While the clock handles the timing of the glitches, you can use MIDI CC knobs from your synths to control the intensity. For example, opening the filter cutoff on your synth can crank up the "tracking error noise" or the "RGB separation distance" on the Pi's shader overlay.

3. Emulating the Sheridan Look on the Pi

When you and your friend start prototyping your effects pipeline, don't look for "generative art" tutorials. Look for tutorials or pre-made code blocks that achieve these exact post-processing techniques:

Horizontal Sync Displacement

This mimics an analog VCR losing its horizontal lock. The shader shifts horizontal lines of pixels left or right based on a noise wave. When a MIDI clock beat hits, you spike the noise, causing the video frame to violently snap sideways and rip apart.

Chromatic Aberration & Color Bleed

Analog tape struggles to keep red, green, and blue light perfectly aligned. Your shader can pull the red channel slightly to the left and the blue channel to the right. You can tie the separation distance to the amplitude of your master audio or a specific MIDI CC message.

Frame-Dropping & Stutter Buffer

A classic digital/analog hybrid glitch. Instead of smoothly playing the CRT video back, the Pi stores the last few frames in a buffer. When a MIDI trigger hits, it freezes a single frame or loops 3 frames rapidly over a 16th note, mimicking a corrupted digital video stream or a skipping disc.

4. Recommended Software Stack for This Workflow

1. The Core Engine: Use openFrameworks (C++) on the Pi. It is lightning-fast on ARM architecture and gives you direct access to video playback and custom GLSL shaders.
2. The Glitch Toolkit: Look up ofxGlitch on GitHub. It is a plug-and-play library for openFrameworks that handles texture glitching, convergence errors, and noise injections out of the box.
3. The Scripting Bridge: If C++ feels too steep for remote collaboration, use Python with opencv-python (for video decoding) and glsl wrappers. It's incredibly easy to share via a quick Git repository, and your friend can write the MIDI parsing logic cleanly.

By using your own CRT footage as the literal fuel and using the Pi's GPU to aggressively chop and displace those frames on every 24th MIDI clock pulse, you're going to get exactly that gritty, broken, industrial atmosphere you are looking for.